March 31, 2026

Armature: Infrastructure for Design Data

Expanding Learning Engineering by Modeling Instructional Design as a Graph of Artifacts and Relations

Amos Glenn
amos@amosglenn.com
University of Pittsburgh

https://github.com/mossglenn/armature

ABSTRACT Learning engineering has built sophisticated infrastructure for capturing what happens during learning experiences. It lacks equivalent infrastructure for capturing the instructional design that produces them. Instructional designers have ways of recording design artifacts. The missing piece is a structured, queryable record of the relations between them and the reasoning behind them: which learning needs generated which objectives, how activities target them, and which assessments measure them. Without these relations, design reasoning remains fragmented within projects and disconnected from the learning data that could inform revision. Armature proposes to fill this gap by treating instructional design as a graph of versioned artifacts and meaningfully named relations. Its goal is to make alignment inspectable, design evolution traceable, and a new class of design intelligence possible before delivery.

1. The Problem

Here is a common scenario: a university identifies that students consistently struggle with a concept on a biology course assessment and asks a learning engineer for help. The engineer's first questions are predictable: *Where in the course are students supposed to develop these skills? Are the learning activities for those skills still aligned with the course objectives? Does the assessment still measure what it was designed to measure?*

Answering these questions means digging through curriculum committee documents, syllabi updated by instructors with varying levels of familiarity with the original objectives, assessment keys maintained by technologists, and emails that someone remembers but cannot find. The information exists, but in pieces scattered across documents, inboxes, and institutional memory. What's gone are the connections between them. These internal gaps form because there is no system for capturing *design data*: the evolving structure of design artifacts and the relationships between them that produce a learning experience.

There are also external gaps between that design data and the associated *learning data*: the structure of the meaningful connections between a learner and their interaction with the learning experience. Learning management systems, xAPI logs, and analytic platforms like OLI/Torus and DataShop capture and analyze learning data in powerful ways. But there is little upstream of learning data; no equivalent system for capturing design data.

The external gap becomes clear when considering where popular and research-based frameworks for using learning and design data fit into learning engineering. This is illustrated in Figure 1 by placing the names of data frameworks on a diagram of the learning engineering process where they would be used. The parts of the cycle that need *learning* data–the section after Creation, through Implementation, and to Investigation—are filled with well-established and widely adopted frameworks and tools. The mostly empty space between Investigation and Creation is where *design* data is needed.

Practitioners have recognized this gap. The LEED tracker (Totino & Kessler, 2024) is a structured approach to recording design decisions and their justifications in tables throughout the learning engineering process. It is a valuable tool for capturing design reasoning that otherwise evaporates, partly because it is low-tech and low-overhead. But its architecture is also its limitation. A table can efficiently record that a decision was made and why, but it remains isolated from other design data (the internal gap) and disconnected from learning data (the external gap). Addressing the gaps within and around design data requires additional infrastructure.

Both gaps point to the same problem: the reasoning that connects problems to objectives to content to outcomes——the instructional design process——has no system of record.

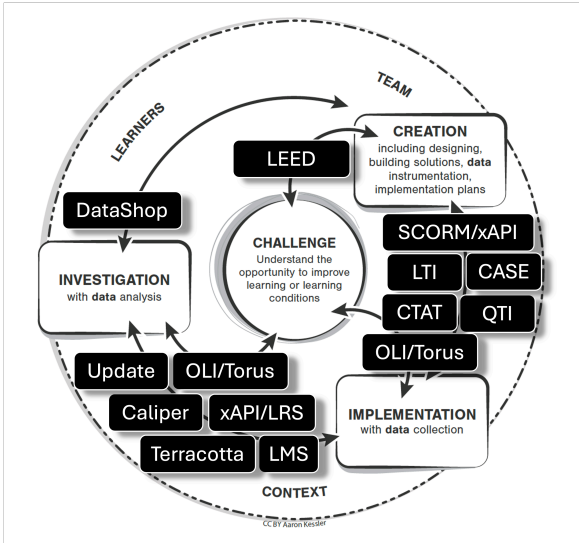

Figure 1: Mapping frameworks for learning data and design data: Established frameworks cluster around Implementation and Investigation. The space between Investigation and Creation—where design data belongs— is conspicuously empty. Frameworks for using learning and design data, represented by black labels, are mapped onto a diagram of the learning engineering process (Kessler et al., 2022), CC BY Aaron Kessler.

2. What Design Data Is

These gaps are not caused by careless practitioners; they are architectural. Every tool in the current ecosystem competently manages its own type of artifact: survey platforms are good at compiling datasets, authoring tools are good at creating content bundles, and learning management systems are good at building quizzes. None of them were designed to model the relationships between those artifacts across the full design process.

What's missing is *design data*: a structured, queryable, versioned record of the instructional design process that produces a learning experience. Design data differs from learning data in who is producing the data, when it is being produced, and what the data represent (see Table 1). Design data is produced by the designer's work during the design process and capture the design artifacts created and the relationships between them.

	Design data	**Learning data**
Who *records it*	Designers	Learners
When *it is recorded*	Accumulated across design	Snapshot at learning
What *it captures*	Artifacts and relations	Actions and performance

Table 1: Design data and learning data operate at different times and capture related processes.

Both kinds of data are valuable on their own. Learning data answers questions about what happened in the learning experience. Design data answers questions about how and why the experience was built. When connected, each becomes more valuable in the context of the other. Outcomes change from a measurement of performance to a diagnosis of the design that produced them. Design decisions change from heuristic decisions grounded in principles to accountable actions based on data (Jonassen, 2008). Neither can close the loop without the other.

Design Artifacts and Design Relations

Design data has two components. **Design artifacts** are the concrete elements produced during design: needs assessment findings, learning objectives, learning activities, assessment items, etc. These are familiar; every designer already works with them.

Design relations are what's currently missing. A design relation is a meaningfully named connection between artifacts that expresses a design decision. For example:

- A learning need *produces* a learning objective
- An objective *is targeted by* an activity
- An objective *is measured by* an assessment item

Relations are equally important to the design structure as artifacts. In instructional design practice today, relations are presumed or implicit connections fractured by documentation. When a designer builds a course, they know which activity targets which objective. Others might be able to infer those connections from context, but when designers leave and courses are revised, those connections evaporate if that relation is not explicitly recorded in ways that can be queried, verified, or analyzed.

Design Process Data

Design data also includes history. When a learning objective is revised, a special type of relation becomes part of the structure: how and why the objective changed. With this historical data, the design structure expresses not just the finished state of the design, but the evolution of the design throughout the process.

These special relations are *design process data*: the record of how artifacts and relations changed over time as the design reached its current state. It answers essential questions about why something was built this way and how it got here. This is deliberately modeled on version control in software engineering, especially the way a git repository records not only the current state of a codebase but every decision that produced it.

3. What Armature Is

Armature is an infrastructure for instructional design. The name comes from the internal framework artists use to support complex sculptures. The unseen structure within the artwork prevents it from

deforming as it grows and maintains its resilience when complete. The armature doesn't replace the artist's tools or the artist's vision; it makes more ambitious sculptures possible. Armature does the same for instructional design.

Armature is not an app or a service, but a framework for encoding instructional design work as design data that is queryable, verifiable, and analyzable. At its core, Armature stores every design artifact and design relation as structured data in a connected graph. Spreadsheets can describe simple relationships between entities: this activity exists and targets that objective. A document or a wiki can describe relationships with prose that cannot be queried: this activity was designed with those principles in mind. Relational databases, commonly the base for LMS data, can capture only predefined relationships that carry no information themselves: this activity targets these objectives and refers to those principles. A graph can describe complex data in queryable ways by treating relationships as first-class entities with their own properties: this activity targets those objectives by applying those principles.

In Armature's design graph, both artifacts and relations are entities with their own names, properties, and history. A simplified illustration of a design graph is shown in Figure 2 . Each dot (or *node*) represents an artifact and each arrow (or *edge*) represents a relation. The structure of the graph represents the instructional design process in a way that can be queried and analyzed.

This makes it possible to record, for a single learning objective, not just its text but:

- What needs assessment data produced it
- Which activities were designed to address it
- Which assessment items measure it
- How those alignments were established and whether they have since changed
- The full history of how the objective itself has evolved

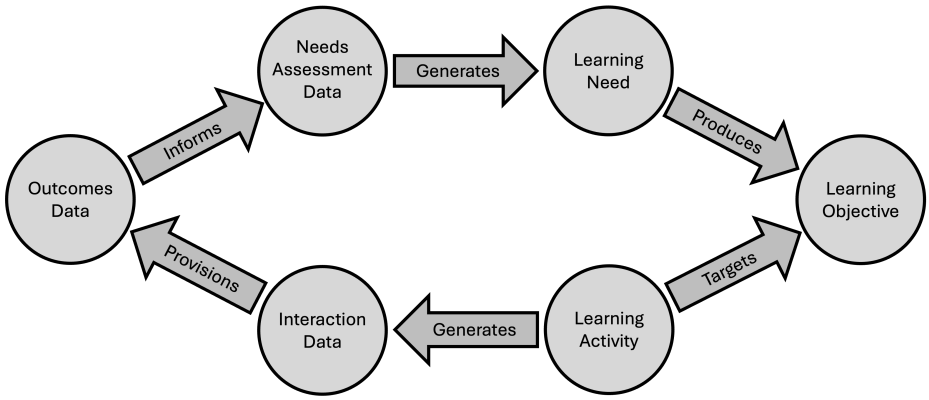

Figure 2: Simplified Design Graph: Each circle represents an artifact, and each arrow represents a relation with a meaningful name. Both are first-class entities in the graph.

The graph can be traversed in any direction. If outcomes data reveal a gap, you can trace backward to find which activities were designed to build that skill. If an objective changes, you can immediately see what activities and assessments need to be reviewed. A spreadsheet, directory, or wiki will store artifacts and notes. A graph models the reasoning that connects them.

Collaboration and Version Control

Armature supports capturing design process data through distributed version control modeled on Git. Everything stored in the graph is immutable and cannot be edited in place. All changes are actually additions to the graph. Creating a new objective creates a new artifact in the graph. Editing that existing objective doesn't change the artifact, but adds a new artifact with a "*new version of*" relation to the existing artifact.

Preserving design process data this way allows branching and merging of design graphs. Designers can copy the design graph to create their own branch for exploring and experimenting without affecting the main design. Teams can work in parallel on different branches without overwriting each other's changes. When an addition or revision is ready, the project lead can safely merge the branches back into the main design. Every merge is an addition to the design graph inscribed with its author, the time, and the reason for the change.

This model of capturing design process data makes design evolution auditable. It also makes it possible for an institution to maintain a clear record of who made each design decision, in response to what evidence, and what it replaced, valuable information usually lost between revisions.

What Armature Is Not

Armature operates at design time, not delivery time. It is not a learning management system, an object repository, or a learning record store. Armature is designer-facing, not student-facing. It is not an adaptive learning engine, an assessment platform, or a content delivery system. Armature is an infrastructure, not a tool. It isn't a content-authoring app, a quiz builder, or a curriculum-mapping tool. It is not a replacement for any existing tool in the design workflow, but new connective tissue supporting all of these things to enable new capabilities that are currently impossible.

4. What a Design Graph makes Possible

Alignment Becomes Inspectable

Currently, alignment is a set of judgment calls made at design time and invisible to everyone afterward. A learning engineer believes their objectives align with assessed needs, and that assessments align with objectives, but there is no mechanism to verify this, especially after multiple people have revised a course over several years.

When design relations are explicit and queryable, alignment is no longer an assertion, but is a property of the design graph that can be inspected. The shape of the graph can tell you about the symmetry between objectives and activities targeting them and the distribution of assessment items. You can identify gaps before delivery rather than diagnosing them from outcomes data afterward.

Iteration Becomes Optimized

When outcomes data reveal a problem, the learning engineer currently faces a manual audit: which objectives relate to this outcome, which activities target those objectives, and how have those activities already been changed. Answering these questions requires collating documents from different systems and relying on whoever remembers.

The design graph turns this into a simple query. The meaningfully named relations trace outcome evidence from the learning experience through every decision anywhere in the design process that contributed to the production of that outcome. Iteration is faster and more precise. Plus the rationale for each revision is preserved, accumulating to aid future diagnostics.

Design Intelligence at Design-Time

Most feedback in instructional design arrives too late. Outcomes data show that students struggled with a concept only after the course was delivered. The learning engineer then works backward to diagnose what went wrong. *Design intelligence* inverts this. When the design graph accumulates artifacts and relations as designers work, some analyses that currently require post-delivery learning data become available during the design process using design data. This means that the current design structure, and possibly the implications of a section being askew, becomes visible to designers as they work.

Consider what a quiz authoring tool built on the design graph can do that a conventional quiz authoring tool cannot:

- Show the designer which objectives currently lack assessment coverage
- Flag when a new item is redundant in a question bank
- Surface the needs assessment data that generated the objective being assessed, giving the designer the context of the original problem beyond the abstracted objective
- Identify when the Bloom's level of a proposed item is inconsistent with the level specified for the objective it measures

None of this requires AI. It requires structured data.

Design intelligence provides practical value immediately. Where learning analytics tells you what happened in a learning experience, design intelligence tells you what the current design structure suggests about the resulting learning experience (Jonassen, 2008). It provides the designer with the

information and insights needed to make better design decisions in real time. A designer who can see coverage gaps while writing assessments closes them before delivery. A designer who can trace an activity back to the evidence of need that generated the objective it targets creates more effective activities. These are workflow improvements that instructional designers will recognize and want. Design intelligence is what makes them possible.

AI Becomes a Design-Aware Collaborator

Design intelligence is also usable by AI as a source of contextual information. AI is already being used in instructional design. Without access to structured design data, it remains oblivious to the design context and can only contribute generic content. With access to the design graph, a generic prompt becomes a bespoke request. A broad prompt like this:

```
Write a quiz about epigenetics.
```

becomes highly specialized:

```
Generate assessment items that measure this specific objective at this Bloom's
level, targeting this known misconception, and avoid overlap with existing
items in the question bank.
```

The graph and the design intelligence it produces turn AI from a content generator into a collaborator that knows what the design requires.

Equally important, AI contributions are subject to the same structural review as human contributions. Every relation added to the design graph must have a meaningful structure and is reviewable, auditable, and reversible regardless of its source. This sidesteps the main barrier to using AI in education: the opacity of AI makes it difficult to trust. When the reason for a decision is modeled in the structure, trust is no longer the basis for acceptance.

From Craft to Engineering

Engineering disciplines improve by studying the relationship between process and outcome. Learning engineering is rigorously studying what works (which learning experiences produce which outcomes). What it cannot yet study is what design practices produce things that work. Until design artifacts and their relations are captured in a common format, it is structurally impossible to ask questions like:

- Are there design patterns that predict poor learning outcomes?
- Do certain approaches to needs assessment produce more durable learning objectives?
- When AI proposes a design artifact and relation, and a human accepts it without modification, is it as reliable as a human-authored artifact and relation?

These questions are currently unanswerable at scale. Design graphs make them possible.

5. How Armature Fits the Existing Ecosystem

Armature is designed to complement existing infrastructure, not compete with it.

- **OLI/Torus** is the most sophisticated existing system for linking design artifacts to learner data. Armature expands on this by modeling the full design process and supports comparison across institutions that may not use the same delivery platform.
- **LearnSphere/DataShop** provides powerful tools for analyzing learner behavior by linking knowledge components to interaction data. Armature operates upstream, modeling the decisions that create the learning activities these systems analyze. Together, they could close the full loop from design decision through learner outcomes and back to informed design revision.
- **Existing standards** handle delivery (LTI, QTI, SCORM), learner data (xAPI, Caliper), and competency frameworks (CASE). None of them model design relations and artifacts. Armature operates in this unoccupied space, referencing these standards without duplicating them. A learning objective in the design graph can link to a CASE competency, and an assessment item can reference its QTI representation.

Design data and learning data are complementary. Learning data tells you what happened. Design data tells you why the experience was built that way. Neither is sufficient without the other. Armature is the missing infrastructure for design data that learning engineering needs to close the loop between design and outcomes.

6. Adoption Paths

Armature will only be adopted if it provides real-world value to adopters. Two natural entry points for adoption are instructional designers and learning scientists. These two groups can have different but supportive adoption paths.

Instructional Designers

Instructional designers will not adopt infrastructure that demands more work before delivering any benefit, and populating a design graph manually would be a significant burden. The best adoption path will satisfy two conditions: design data should accumulate as a byproduct of the designer's existing work, and the tools that produce design data must provide immediate, recognizable value on their own terms.

Satisfying these conditions means addressing one of Armature's biggest adoption challenges: the lack of open standards in instructional design. Many instructional design tools use proprietary formats that are often undocumented and legally protected against reverse engineering. If artifacts created

elsewhere cannot be exported, Armature cannot rely on easy import or migration pipelines as part of the adoption strategy.

These constraints point toward a tools-first strategy: design new tools that solve real workflow problems for instructional designers, deliver design intelligence they can see and use, and quietly encode their work into the design graph as a consequence of normal use. For example, a quiz authoring tool that shows which objectives lack assessment coverage, flags redundant items, or surfaces the needs assessment data behind an objective might be a good place to start. The designer gets a better authoring experience; the graph gets richer without additional effort.

Git was adopted in software development along similar lines. Developers were not adopting an immutable history of their code. They adopted Git because it made collaboration easier. The deeper structural benefits of the immutable history (e.g., auditability, branching, and distributed contribution) became possible once data had accumulated simply by using Git.

The same logic can apply to Armature. Individual designers adopt a tool because it improves their immediate work. Over time, teams using tools on the same graph gain something new: a shared, queryable record of design decisions they can inspect, question, and build on together. As adoption grows, institutions and researchers gain something new: comparable design data across courses and programs alongside the learning data that's been available for some time, closing the learning engineering loop.

Each stage depends on the previous one generating data worth having. That dependency and the lack of open standards argue for starting with genuinely useful tools that are not just thin graph interfaces, and letting the infrastructure prove its value over time.

Learning Scientists

The tool-first path is not the only entry point. Answering a specific research question around instructional design practices may not need a complete implementation of Armature, only the relevant slice of the graph. Take, for example, scientists investigating whether the quality of a needs assessment predicts the durability of learning outcomes. That study requires structured data that connects learning needs to objectives and objectives to outcomes, but it does not require data on learning activities, assessments, coverage, or alignment. A lightweight data-collection instrument and a constrained graph schema could be sufficient to generate comparable design data for a small set of courses.

Again, both the scientist and the design graph benefit from this strategy. The scientist gets a comparable structure for their data that others can replicate, extend, or challenge. The graph gets evidence to inform the development of its schema and to improve its model of relationships between design artifacts.

Mutually Reinforcing Paths

These two adoption paths do not conflict, but reinforce each other. The tool-first path uses infrastructure to produce data as a byproduct of design work, while the data-first path inverts this by using data to produce infrastructure as a byproduct of research. Both paths lead to the same place: a growing body of structured design data that can be studied alongside learning data. Neither depends on the other going first. A researcher's schema becomes a precedent that informs tool design. A tool's accumulated data becomes a population that researchers can study. Any progress on one path advances the other.

Conclusion

Learning engineering has focused on understanding what happens during learning experiences. The infrastructure for capturing learning data is mature and continuing to develop. What does not yet exist is comparable infrastructure for capturing design data, the structure of design artifacts and design relations that produce learning experiences.

Without design data, the impact of design on outcomes remains opaque. We can measure what happened, but we cannot systematically study why experiences were designed the way they were, trace an outcome back to the design decision that shaped it, or accumulate design knowledge in a form that improves future practice.

Armature proposes to fill this gap by treating instructional design as structured data: versioned artifacts and the meaningfully named connections between them. The result is not a new authoring tool or a replacement for existing systems. It is the missing infrastructure for capturing the design data that learning engineering currently overlooks.

> **Armature is an active research and development project.**
>
> Questions, critique, and collaboration are welcome. You can find project documentation and updates at https://github.com/mossglenn/armature.

AI DISCLOSURE

While writing this manuscript, the author used Claude (Anthropic) to assist in clarifying ideas and drafting manuscript text. All content was revised by the author and all language, arguments, and claims reflect the author's own judgment.

Iterative development of Armature architecture was supported by Claude (Anthropic) by serving as a sounding board, drafting schema and code, and automating project documentation. All conceptualization, framing, and design intent originated with the author. All artifacts were reviewed and approved by the author.

References

Jonassen, D. H. (2008). Instructional Design as Design Problem Solving: An Iterative Process. *Educational Technology*, *48*(3), 21–26.

Kessler, A., Craig, S. D., Goodell, J., Kurzweil, D., & Greenwald, S. W. (2022). Learning Engineering Is a Process. In J. Goodell & J. Kolodner (Eds.), *Learning Engineering Toolkit: Evidence-based Practices from the Learning Sciences, Instructional Design, and Beyond: Learning Engineering Toolkit: Evidence-based Practices from the Learning Sciences, Instructional Design, and Beyond* (1st ed., pp. 29–44). Routledge. https://doi.org/10.4324/9781003276579

Totino, L., & Kessler, A. (2024). ``Why Did We Do That?" A Systematic Approach to Tracking Decisions in the Design and Iteration of Learning Experiences. *The Journal of Applied Instructional Design*. https://doi.org/10.59668/1269.15630